# PhyAuth-RF: Live Demo and Viva Guide

How to run the demonstration in front of the faculty, what to point at, what to say, and what you must know (technical and supporting). Keep this beside the laptop.

| Total time | Hardware on the table | Screens to keep open |
|---|---|---|
| About 10 minutes of demo + 5–10 minutes of questions | Receiver node (ESP32 + SRX882 + LED + buzzer), transmitter node (ESP32-S3 + STX882), QC5681 remote, laptop | Dashboard (localhost:3000): Live, Attack lab, Smart Lock, Model lab · slide deck · printed report |

## 1. Team roles (decide before the review)

| Role | Responsibility |
|---|---|
| **Presenter** | Speaks the story (problem, idea, results); keeps time; hands questions to the right person. |
| **Hardware operator** | Holds the remote and presses buttons at the right moment and distance; enters the lock pattern. |
| **Dashboard operator** | Switches dashboard pages, launches the clone and unknown-device transmissions, points at numbers. |
| **Q&A lead** | Answers technical questions (DSP, ML, firmware); has the report open to back up every number. |

Rotate roles in the rehearsal so that every member can explain every step; faculty often ask a question to a specific member.

## 2. Set-up checklist (15 minutes before)

| # | Check | How / expected result |
|---|---|---|
| 1 | Laptop on charger, sleep disabled | System Settings → Lock Screen / Battery: never sleep during the review. |
| 2 | Both boards connected by USB-C | Receiver: /dev/cu.usbserial-0001 · Transmitter (ESP32-S3, 'COM' port): /dev/cu.usbmodem1101 |
| 3 | Start the back-end | In the project folder: `.venv/bin/python -m backend.main` |
| 4 | Start the dashboard | `npm --prefix frontend run dev`, then open http://localhost:3000 |
| 5 | Connect hardware | Hardware page: connect the receiver. Attack lab: connect the transmitter, then press **AUTO 0** (no automatic presses). |
| 6 | Top bar is all green | Backend ● · Receiver usbserial-0001 · Model **20261021-224341** · Lock **locked**. The model must be the same on ESP32 and laptop. |
| 7 | Place the remote | Antenna of the remote 0.75–1 m from the receiver antenna. Never closer than 0.5 m (receiver saturates). |
| 8 | Two test presses | Press ON twice: both AUTHENTIC, score below 1. Press OFF once: AUTHENTIC. If OFF is close to 1, keep to 0.75–1 m. |
| 9 | Lock pattern ready | The 9-press ON/OFF pattern written on paper for the hardware operator only (do not show it on screen). |
| 10 | Backup ready | Slide deck with screenshots open in a second window; Hardware page → **Use simulator** if the radio misbehaves. |

## 3. Demo script, step by step

Each step says what to **do**, what to **show** on screen or hardware, and what to **say** (read the italic lines naturally, not word for word).

| **STEP 1** | **Introduce the problem with the hardware on the table** | 1 min |
|---|---|---|
| **DO** | Hold up the remote, then point to the receiver node and the transmitter node. | |
| **SHOW** | Remote (QC5681, EV1527), receiver (ESP32 + SRX882 + green LED + buzzer), transmitter (ESP32-S3 + STX882). | |
| **SAY** | *"This remote sends a fixed 24-bit code at 433.92 MHz. Anyone who records the code can replay it, because the receiver checks the message, not the transmitter. We authenticate the transmitter itself, from tiny timing imperfections in how it transmits, using a five-dollar ESP32."* | |

| STEP 2 | A genuine press is accepted | 1.5 min |
|---|---|---|
| **DO** | Dashboard on **Live**. Hardware operator presses **ON** once at about 1 m. | |
| **SHOW** | Verdict **Authentic**, score below 1 (threshold is 1), frames fused 4, clock T ≈ 335 µs, **Decided on ESP32**.<br>Point at the pulse train and the residual bars: orange HIGHs below zero, blue LOWs above zero. | |
| **SAY** | *"The ESP32 timestamped every edge with its 240 MHz cycle counter, 4.17 ns resolution, decoded the code, extracted 11 timing features and compared them with the remote's stored profile. The score is below 1, so it is authentic. The bars show the receiver shortening every HIGH and stretching every LOW; our decoder is immune to that bias."* | |

| STEP 3 | The other button is recognised too | 0.5 min |
|---|---|---|
| **DO** | Press **OFF** once. | |
| **SHOW** | Different code (936764) but the same device, FOB_A, Authentic. | |
| **SAY** | *"The features never use the bits, so the fingerprint belongs to the transmitter, not to the code. One model covers every button."* | |
| **NOTE** | If OFF scores just above 1, press again from 0.75–1 m and say: "A single press can be rejected; the measured burst error is about 1 %." | |

| STEP 4 | Clone attack: same code, same clock, still rejected | 1.5 min |
|---|---|---|
| **DO** | Open **Attack lab**. Victim = **FOB_A**. Click **Launch clone attack**.<br>Optionally press the remote again right after, to contrast. | |
| **SHOW** | The ESP32-S3 transmits FOB_A's exact code 0x936768 with FOB_A's clock period.<br>Verdict **Spoof suspected**, score well above 1, on both ESP32 and laptop. | |
| **SAY** | *"This is the attack a normal receiver cannot stop: identical code, identical clock period. Our receiver rejects it because the edge shape and jitter of a different transmitter do not match. In testing, 30 out of 30 clone bursts were rejected."* | |

| STEP 5 | Unknown transmitter is rejected | 0.5 min |
|---|---|---|
| **DO** | In Attack lab click **PROFILE 3**, then **PRESS**. | |
| **SHOW** | Verdict **Unknown device** (code 9b0d2a was never enrolled). | |
| **SAY** | *"This is open-set recognition: the model was never trained on this transmitter and it does not force it into a known class. 30 out of 30 unknown bursts were rejected."* | |
| **NOTE** | Afterwards press **AUTO 0** so the transmitter stays silent during the lock demo. | |

| STEP 6 | Two-factor smart lock | 2.5 min |
|---|---|---|
| **DO** | Switch to **Smart Lock**. Hardware operator enters a **wrong** pattern of 9 presses.<br>Then enters the **correct** 9-press pattern.<br>Wait 15 s, or click **Lock now**. | |
| **SHOW** | Wrong pattern: dots fill, then a single failure at the end (no hint which press was wrong); LED fast-blinks; failures 1 of 3.<br>Correct pattern: padlock opens, **UNLOCKED**, factor 1 ‘genuine’, green LED on; auto-relock after 15 s. | |
| **SAY** | *“Two factors from one unmodified remote: something you have, the remote’s RF fingerprint, and something you know, the ON/OFF pattern. The attempt is judged only at full length, so watching does not reveal the pattern. A clone press spoils the attempt even with the right pattern, and three failures cause a lockout that doubles each time. The logic runs on the ESP32; the pattern is in its flash.”* | |
| **NOTE** | Our LED is a self-flashing type, so it blinks instead of glowing steadily, and the 5 V buzzer is quiet on a 3.3 V pin. Say so up front; both are listed as hardware fixes (plain LED, NPN driver). | |

| STEP 7 | Evidence: model and numbers | 1.5 min |
|---|---|---|
| **DO** | Open **Model lab & deploy**; then show the Results slide. | |
| **SHOW** | ROC curve, confusion matrix, feature importance, embedding map (clone × outside every cluster).<br>Edge model: 963 parameters, 7 KB header, export and flash. | |
| **SAY** | *“On held-out real captures: 100 % closed-set accuracy, frame-level EER 0.49 %, burst-level EER 0, all clone and unknown bursts rejected. Four frames are fused per press, which takes about 172 ms. Clock period and jitter carry most identity.”* | |

| STEP 8 | Close | 0.5 min |
|---|---|---|
| **DO** | Return to the title or summary slide. | |
| **SHOW** | Thank-you slide. | |
| **SAY** | *“A $5 microcontroller behind a $1 receiver can tell the genuine remote from a perfect copy of its code, and open the door only for the right remote and the right pattern. We are happy to take questions — and you are welcome to press the remote yourself.”* | |

# 4. Points to mention (and the numbers behind them)

| Point | Number / fact to quote |
|---|---|
| Problem | Fixed-code 433 MHz remotes (EV1527) can be recorded and replayed; the receiver checks only the code. |
| Idea | Authenticate the **physical transmitter** from its timing imperfections (physical-layer authentication). |
| Measurement | Edge timestamps from the ESP32 cycle counter: 240 MHz → **4.17 ns**; no SDR, no ADC. |
| Signal | OOK at 433.92 MHz; 0 = 1T H + 3T L, 1 = 3T H + 1T L; sync 1T + 31T; T ≈ 335 µs; frame = 128T = 42.9 ms. |
| DSP contribution | Receiver bias up to 0.33T; clock from pair period (= 4T) + 31T sync gap; pair-based bit decisions. |
| Features / model | 11 code-invariant features → EdgeMLP 11–32–16 (**963** parameters, **7 KB**) → Mahalanobis test. |
| Decision | Median of 4 frame scores ≤ 1 → AUTHENTIC; else SPOOF (enrolled code) or UNKNOWN. |
| Results | Accuracy 100 % · frame EER 0.49 % (AUC 0.99996) · burst EER 0 % · clone 30/30 · unknown 30/30 rejected. |
| Fusion | FRR 45.6 % with 1 frame → 1.1 % with 4 frames; latency ≈ 172 ms. |
| Range | Reliable 0.75–1.25 m; < 0.5 m saturates, ≥ 1.5 m loses short pulses. |
| Lock | Pattern 4–12 presses (demo 9); 3 failures → lockout 30 s × $2^{k}$; auto-relock 15 s; NVS flash. |
| Cost / power | Verifier ≈ Rs. 650; whole kit ≈ Rs. 2,000; < 1 W. |
| Engineering | 35 automated tests; C on the ESP32 checked against Python (parity test). |

# 5. Must-know technical questions

## Basics and communication theory

### Q. What is RF fingerprinting / physical-layer authentication?

Identifying a transmitter from unintended analogue imperfections in its signal (clock error, jitter, edge/turn-on shape) instead of from the data it sends. The imperfections come from component tolerances and are hard to copy with different hardware.

### Q. How is this a Digital Communication Systems project?

Binary ASK/OOK modulation and non-coherent envelope detection (SRX882 superhet), PWM line coding (EV1527), symbol-clock and frame synchronisation, decision thresholds and receiver distortion, time quantisation (4.17 ns), binary hypothesis testing with FAR/FRR/ROC/EER, and repetition combining (4-frame fusion).

### Q. Explain the EV1527 frame.

24 bits = 20-bit address + 4-bit key. Each bit is 4T: 0 = 1T HIGH + 3T LOW, 1 = 3T HIGH + 1T LOW. Sync = 1T HIGH + 31T LOW. Frame = 24×4T + 32T = 128T ≈ 42.9 ms at T = 335 µs. ON = 0x936768, OFF = 0x936764.

**Q. Why was decoding the real remote difficult?**

The SRX882 slicer threshold lags the envelope, so every HIGH is shortened and every LOW lengthened, by up to a third of T near the antenna. A 3:1 pattern at T then fits a grid at 2T/3, so a histogram-based clock estimator locked onto the wrong clock and decoded nothing.

**Q. How did you solve it?**

The bias cancels inside a HIGH+LOW pair, so the pair period P = 4T is unbiased. The 31T sync gap fixes the number of units per pair uniquely, giving T = P/4. Bits are decided by which half of the pair is longer, which is also immune to the bias.

**Q. How do you measure time so precisely?**

A GPIO interrupt on every edge reads the CPU cycle counter (240 MHz, 4.17 ns per count) into a lock-free ring buffer. Frames are cut after 4 ms of LOW; glitches under 80 µs are merged.

**Q. Why does distance matter?**

The slicer bias depends on signal strength. Below about 0.5 m the receiver saturates; beyond about 1.5 m the 1T pulses are lost in noise. 0.75–1.25 m is the reliable window with spring antennas.

## Features, model and decision

**Q. What are the 11 features? Which matter most?**

Clock period T, bit-period jitter, four duty-cycle errors (short/long HIGH and LOW), HIGH jitter, first-edge and trailing-edge residuals, drift per bit, sync ratio. Shuffling clock T raises EER by about 27 points and pair jitter by about 16 points; sync ratio contributes almost nothing.

**Q. Why can the clone not pass if it copies the clock period?**

Clock period is only one feature. The clone's power amplifier and timebase give different edge asymmetry, first-edge residual and jitter; in the feature plot it is clearly separated from the remote.

**Q. Why a small MLP and not a CNN on the chip?**

The features are already engineered, so 11→32→16 (963 parameters, about 1.1 k multiply-accumulates) is enough and fits in 7 KB. A larger host model (1D ResNet + MLP, 21 k parameters) runs on the laptop for comparison; it is not better at the operating point.

**Q. What is the Mahalanobis distance and why Ledoit–Wolf?**

Distance to the device's centroid scaled by the inverse covariance of its embeddings (a whitened / Gaussian likelihood distance). With few samples in 16 dimensions the covariance is ill-conditioned, so it is shrunk towards a scaled identity (Ledoit–Wolf).

**Q. Why not just use softmax classification?**

Softmax always picks one of the known classes, so an unknown transmitter or a clone would be labelled as some enrolled device. A distance threshold per device gives open-set rejection.

**Q. How is the threshold chosen?**

From genuine data only: 97th percentile of genuine validation distances × 1.3 margin. Attack data are never used for tuning, so attack results are not optimistic. Score = distance / threshold; ≤ 1 passes.

**Q. Define FAR, FRR, EER, AUC and give your values.**

FAR = impostors accepted / impostor attempts; FRR = genuine rejected / genuine attempts; EER = rate where they are equal; AUC = area under the ROC. Frame EER 0.49 %, AUC 0.99996; burst EER 0 %, FAR 0 %, FRR 0 % on the test set.

**Q. Why fuse four frames?**

A held button repeats the frame; the median of 4 scores averages out per-frame noise (repetition / time diversity). Genuine rejection falls from 45.6 % to 1.1 % at about 172 ms latency, with no loss in attack rejection.

**Q. How did you split the data?**

Temporal split, not random: later presses are held out. 712 training, 137 validation, 152 test genuine frames; 420 impostor frames (clone and unknown) used only for testing. The OFF button is merged into FOB_A as a variant.

**Q. What is the liveness check and why is it off?**

A remote powers up on each press, so its clock drifts during the first frames (median −1031 ppm); the always-on clone shows about 5 ppm. The press-to-press spread (sd 526 ppm) is too large for a safe window, so the check switches itself off rather than reject genuine presses.

### Firmware, lock and system

**Q. How does the model get onto the ESP32, and how do you know it is correct?**

The exporter writes scaler, weights, centroids, inverse covariances and thresholds into phyauth_model.h (7 KB). A parity test compiles the C code on the laptop and checks it gives the same features and scores as Python. The model version is shown in the dashboard top bar for both sides.

**Q. How does the lock work?**

A C state machine: DISABLED, LEARN, LOCKED, UNLOCKED, LOCKOUT. Every press counts; the attempt is judged at full length; a fingerprint-rejected press spoils it; 3 failures → 30 s lockout doubling up to 8×; 15 s auto-relock; 30 s idle clears an attempt without penalty; the pattern is stored in NVS flash. 10 unit tests cover these rules.

**Q. What if someone records the remote and builds a perfect timing copy?**

A programmable waveform source could in principle imitate the timing; this raises the attack from a $5 cloner to a targeted, instrumented effort. The lock also needs the secret pattern, and liveness can be enabled for remotes with consistent start-up drift.

**Q. What happens with a different receiver?**

The receiver's own distortion is part of the measurement, so a new receiver needs re-enrolment (about 30–40 presses per button and distance).

**Q. Why ESP32-S3 for the transmitter?**

Its RMT peripheral generates pulses with 100 ns resolution; a whole press is sent as one continuous waveform (separate calls left a 2 ms gap that corrupted the last bit).

## 6. Must-know supporting points

### Honest limitations (say them before you are asked)

- Only **one genuine remote**; EMU_1/EMU_2 are emulated and differ by programmed timing. The security claim rests on genuine remote vs. clone and unknown transmitter.
- Small test sets (27 genuine, 60 attack bursts): zero errors does not prove a zero error rate.
- Range 0.75–1.25 m; data from two days indoors; temperature and battery ageing not studied.
- The live re-test of the newly retrained OFF button is part of this demo.

### Impact and relevance

- Use cases: garage doors and gates, smart door lock, alarm panels and sensors, smart plugs, industrial remotes, teaching lab.
- SDG 9 (innovation), 11 (safe communities), 12 (retrofit instead of replace), 7 (< 1 W), 4 (education).
- Cost: verifier ≈ Rs. 650 vs. SDR testbeds costing hundreds to thousands of USD.

- Privacy: only timing data of the user's own devices; the clone emulator is used only on our own remote.

### What not to say

- Do not say "unhackable" or "100 % secure"; say "rejected every clone and unknown burst in our tests".
- Do not claim we tested many remotes; say "one genuine remote plus emulated and attacking transmitters".
- Do not quote the on-chip inference time in microseconds; we report decision latency (≈ 172 ms, set by frame air time).

## 7. If something goes wrong

| Symptom | Quick fix | What to say |
|---|---|---|
| No decision appears | Move the remote to 0.75–1 m, antenna vertical; press and hold a little longer. | "The receiver needs a clean signal window; too close saturates it." |
| Dashboard frozen | Wait 8 s (watchdog reopens the port) or Hardware page → Disconnect / Connect. | — |
| ESP32 and laptop disagree | Check both show model 20261021-224341 in the top bar. | "Both sides must run the same model version." |
| Genuine press rejected once | Press again from the same spot. | "Single-press FRR is about 1 %; this is that case." |
| Transmitter does nothing | Attack lab → Disconnect / Connect; replug USB-C (COM port). | — |
| Radio unusable in the room | Hardware page → **Use simulator**, or present the screenshots in the deck. | "Here is the same flow recorded on our hardware." |
| Lock in LOCKOUT | Wait for the timer or remove and re-set the pattern in Settings. | "That is the brute-force protection." |

**Tip:** invite a faculty member to press the remote for step 2 and to try a wrong pattern in step 6 — a live, hands-on test is the most convincing part of the demo.